CWI

Research institute for mathematics & computer science in the Netherlands

SESSION 1 · FUNDAMENTALS OF AI AGENTS

AI Agents in Practice

M. Bilgehan Ertan · Marten van Dijk

Computer Security group, CWI Amsterdam

Tuesday 3 November 2026 · 10:30–12:30 · Euler room, CWI

0:00–0:07

Welcome

CWI

Research institute for mathematics & computer science in the Netherlands

Welcome

This course

- Five sessions on *using* AI agents in research and technical work
- No ML background needed
- About 30 minutes hands-on in every session

Today

- What an agent is
- What you can use it for, and what goes wrong
- What happens when you type a request
- Mental models that predict its behaviour
- Your agent installed, a first project, and a take-home task

Goal for today. Leave with a correct mental model of what an agent does with your request, and with a first task of your own to try.

Five sessions



- Tuesdays 10:30–12:30. Sessions 1–3 are the core practical sessions. Session 4 is for the curious: how models are trained, and safety. Session 5 is yours.
- Between Sessions 2 and 3 (three weeks) you use an agent on your own work.

Who is here?

Which have you used? (hands up)

- Chat assistants (ChatGPT, Claude, Gemini)
- An agent in the terminal (Claude Code, Codex, Gemini CLI, Antigravity CLI)
- An agent in your editor
- None of these yet

What is your research area?
In a few words, from a few of you.

Is your agent installed and logged in?
If not, tell us now, so we know who needs help in the installation block.

One rule for the whole course

You are responsible for every claim, citation, result, and line of code you put your name to, whether or not an agent produced it.

- The agent does the work you delegate. The accountability stays with you.
- So this course teaches you to *get* output and to *check* it.

Adapted from the AI-use policy of CMU 11-768 *AI Agents* (Fried and Neubig, 2026).

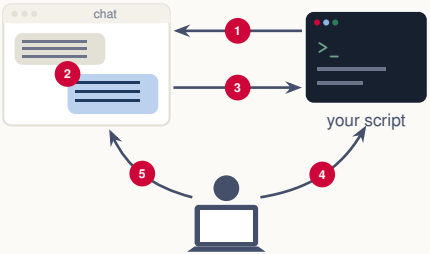
0:07–0:12

What is an agent?

Same problem, chat or agent: “my plot looks wrong”

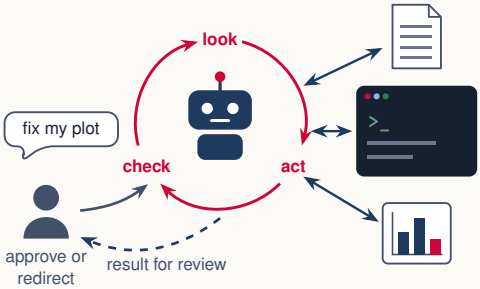
Chat: you do every step

1 paste 2 read fix 3 copy back 4 run 5 report



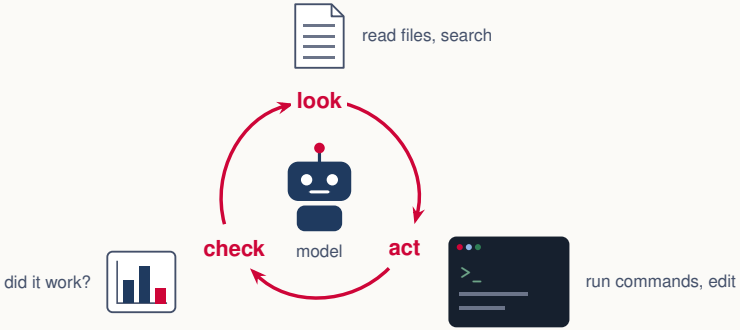
Agent: it acts, you approve

you hand over the task once



Same kind of model. An agent acts on its own until it finishes or asks you.

An agent, in one sentence



An agent is a language model that can use tools (read files, run commands, search) and repeats **look** → **act** → **check** until it thinks the task is done, or stops to ask you.

0:12–0:16

What can I use it for?

What can I use it for? Writing and research



Drafting a paper

drafts and revises sections, fixes compile errors and notation

You check: every sentence. You stay the author.



Literature review

finds and organises related work, and can help check citations

You check: each source exists and says what you claim. You sign off.



Proofs and maths

checks steps, tries counterexamples, can write Lean

You check: every step, and that the statement is the one you meant



Review and rebuttal

reviews your paper as a reviewer would, and drafts the rebuttal

You check: your own work only, never a paper you review. Every promise in the rebuttal.

You stay responsible: the agent drafts and checks, you decide what goes out under your name.

What can I use it for? Proposals, teaching, data, code



Proposal text

structure, consistency, the call's requirements

You check: the call itself, and every promise



Slides, exercises

turns notes into slides, exercises and solutions

You check: solve the exercises yourself



Data and figures

analysis and plots, including user-study data

You check: the numbers.



Code, small tools

explains code, builds a project web page

You check: what it changed, and that it works

Agents can carry out real multi-step work: worth learning to use well, and to check.

Agents resolving real GitHub issues: Yang et al. (2024); SWE-agent team (2026). Literature research with agents: Hadfield et al. (2025).

0:16–0:26

What goes wrong, and what it may do

What goes wrong



Invents a reference

a plausible title, authors, year: but the paper does not exist



Says “done” when it isn’t

a plausible result that misses cases you care about



Confidently wrong, or agrees with you

even when you are the one who is wrong

Agents are capable and confidently wrong in uneven ways. How fluent the answer sounds does not tell you which case you are in.

Kalai et al. (2025); Anthropic (2026d); Sharma et al. (2023).

Sort it: what may an agent do on its own?

1 Autonomous	2 Ask first	3 Never
A Analyse my data and report the results in my paper	**B** Email my co-authors	**C** Make my code and data public on GitHub
D Install the software it thinks it needs	**E** Do a literature review for my paper	**F** Summarise a paper I am reviewing under confidentiality

If it can act, what should it be allowed to do? *what made you hesitate?*

Format adapted from CMU 11-768, Lecture 1 (Fried and Neubig, 2026). Scenarios are ours.

What decides the column

		Undo easily?	Affects others?	Leaves your control?
A	Report data results in my paper	yes, until submitted	later, readers	possibly, to the provider
B	Email my co-authors	**no**	**yes**	**yes**
C	Code, data public on GitHub	**no**: copies, forks stay	**yes**: co-authors, people in the data	**yes**: public
D	Install software it needs	mostly	usually no	runs code from the web
E	Literature review	yes	later, readers	possibly, to the provider
F	Summarise a submission	**no**	authors	**yes**, to the provider

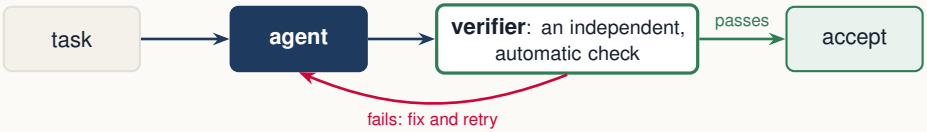
Separately, file contents and tool output in the transcript go to the model provider unless the model runs locally.

- The *action* decides: B and C cannot be taken back. C can also leak private data.
- A and E look safe. The risk is trusting them unchecked. F: venue policy and where the model runs (Sessions 2, 5).
- D runs code from the web that you have not read (Session 2). Today's setup asks first.

There is no oracle. Whether it may act on its own depends on your scientific integrity and on how you verify. With a robust way to check, you can let it run.

Plausible but false statements: Kalai et al. (2025). Public forks stay public: GitHub (2026).

Autonomy, earned by verification



Code

the tests you wrote. The agent must not edit them.

a verifier exists

Proof

Lean checks every step. You check it is the statement you meant.

a verifier exists

Numbers

recomputed by a second script. By hand, it is your review.

a verifier exists

Writing

literature review, prose: no automatic verifier. You are the verifier.

ask first, review all

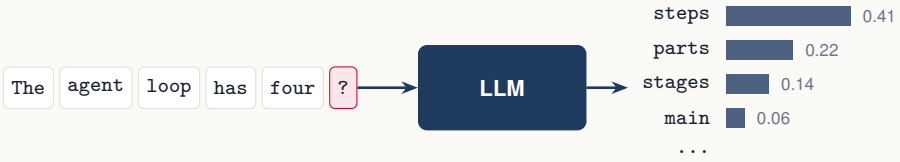
Autonomy follows verification: the more robust the check, the more you can let it run.

Lean: de Moura and Ullrich (2021).

0:26–0:48

From LLM to agent

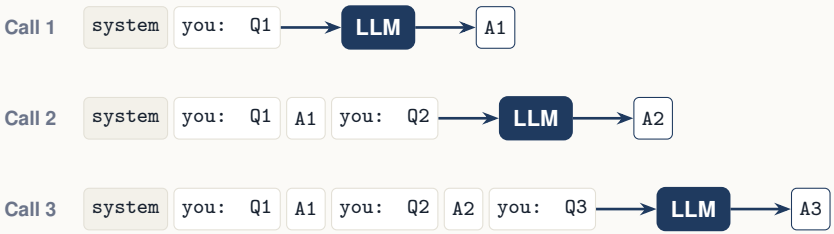
Step 1: a language model predicts the next token



- Text is split into *tokens* (words or word pieces). The model outputs a probability for every possible next token.
- One token is sampled, appended, and the model runs again. That is all generation is.
- Everything today, including tool use, is built from this one operation.

Probabilities are illustrative. How models are trained to do this: Session 4.

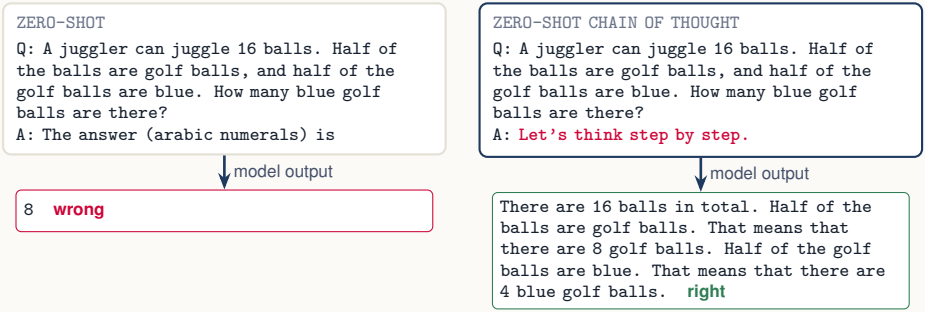
Step 2: chat is a stateless function of the transcript



- The model keeps nothing between calls. The *application* resends the whole transcript every time.
- The **system prompt** comes first: instructions the tool or provider sets (role, rules, "never . . . "). It shapes behaviour a lot.
- "It remembers what I said" means: it is still in the transcript.
- Longer conversations cost more per call, because everything is read again. Providers cache the repeated part, so re-reads are cheaper. They still cost something.

Caching: Anthropic (2026a).

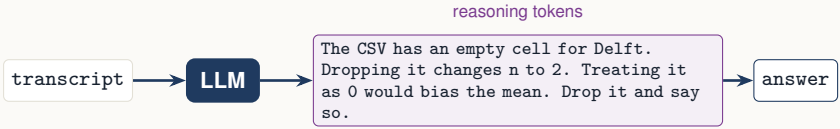
Before reasoning models: “Let’s think step by step”



- 2022: worked examples that show the reasoning steps help (Wei et al., 2022). One sentence without examples also helps: on MultiArith, 17.7% → 78.7% correct (InstructGPT `text-davinci-002`) (Kojima et al., 2022).
- Reasoning models are now *trained* to do this before they answer. That is Step 3.

Example from Kojima et al. (2022), Fig. 1 (2022 results). Few-shot chain of thought: Wei et al. (2022).

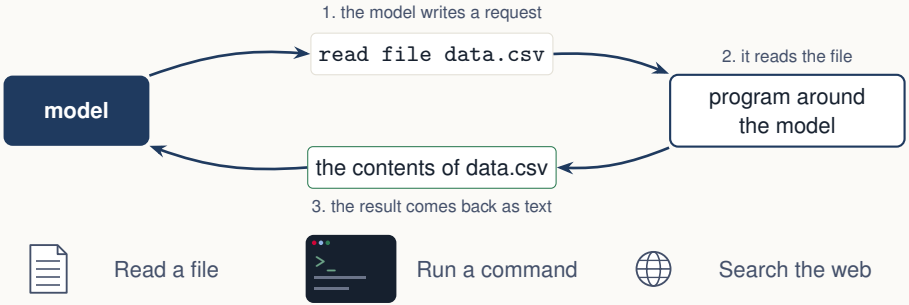
Step 3: reasoning tokens



- Before answering, the model can generate intermediate “thinking” text. Those tokens become context for what follows.
- Reasoning models such as OpenAI o1 and DeepSeek-R1 are trained with reinforcement learning to reason this way.
- Reasoning tokens cost time and money.
- Reasoning text is output. Do not assume it is a faithful log of how the model decided. Check the results themselves.

OpenAI (2024); DeepSeek-AI (2025). Unfaithful reasoning: Turpin et al. (2023). The example reasoning text is illustrative.

What is a tool?



The model can only write text. A *tool* is an action the program around it offers: the model writes a request, the program carries it out, and the result comes back to the model.

Step 4: tools are described to the model in JSON

```
BASH_TOOL = {
  "type": "function",
  "function": {
    "name": "bash",
    "description": "Execute a bash command",
    "parameters": {
      "type": "object",
      "properties": {
        "command": {
          "type": "string",
          "description": "The bash command to execute"
        }
      },
      "required": ["command"]
    }
  }
}
```

- A *tool definition*: name, description, and a JSON schema for the arguments.
- The harness sends it with every request, next to the transcript.
- The only tool of mini-swe-agent, a minimal agent (appendix). Our Claude Code transcript (later) offered 56 tools, 28 built in.
- To the model, a tool is just this text.

Reformatted from `src/minisweagent/models/utils/actions_toolcall.py` (SWE-agent team, 2026), MIT licence (notice at the end of the deck).

Step 5: a tool call is just tokens

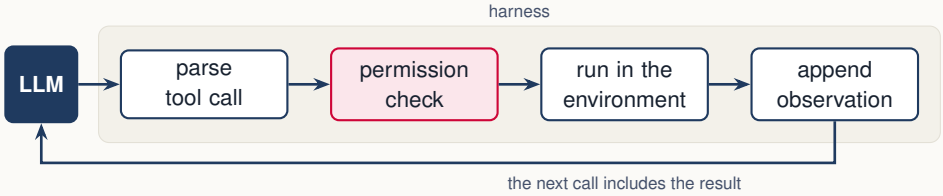
```
{ "role": "assistant",
  "content": "Let me look at the data first.",
  "tool_calls": [{
    "id": "call_1",
    "type": "function",
    "function": {
      "name": "bash",
      "arguments": "{\"command\": \"head measurements.csv\"}"
    }
  }]
}
```

- The model *cannot run anything*. It writes text in a format that requests an action.
- Models are trained to emit this format, and the API returns it as structured data.
- Nothing has happened on your machine yet.

An agent's actions are tokens. Whether they become real actions is decided outside the model.

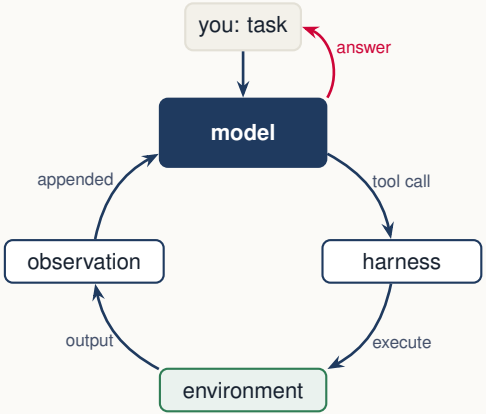
Illustrative message in the OpenAI-compatible format that mini-swe-agent requests through LiteLLM (SWE-agent team, 2026). The token-level format differs per model (Session 4).

Step 6: the harness executes



- The *harness* is the program around the model: Claude Code, Codex, Gemini CLI, mini-swe-agent.
- It decides whether to run the action, runs it, and adds the result to the transcript for the next call.
- Permission prompts, sandboxes and logs live here, in the harness.

Step 7: the loop



- This is the look → act → check cycle from the start, now with the real parts.
- Repeat until the harness stops. In Claude Code: an answer without a tool call, or a limit.
- ReAct named the pattern: interleave *reasoning* with *actions*, and let observations steer the next step.

An agent is a model in a loop with tools and an environment.

Yao et al. (2022). Framing after Schluntz and Zhang (2024) and Fried and Neubig (2026).

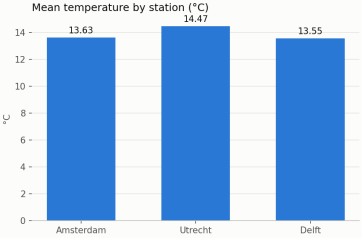
A real transcript, annotated

Task: *"Compute the mean temp_c per station from measurements.csv, then save a bar chart of those means as means.png."* Claude Code 2.1.302, 26 Oct 2026. CSV with 9 data rows, one empty cell. Condensed.

	The model wrote a tool call	The harness ran it and returned
1	`Skill: dataviz`	chart-style instructions installed on this machine
2	`Bash: head -20 measurements.csv && wc -l ...`	the file: Delft 2026-10-02 is empty
3	`Bash: grep ... palette.md`	colour codes from the skill
4	`Write: station_means.py`	"File created" (the script skips empty cells)
5	`Bash: python3 station_means.py`	the means, and "Skipped 1 row(s) with missing temp_c"
6	`Read: means.png`	*the image itself*
7	`Edit: labels {v:.1f} → {v:.2f}`	"updated"
8	`Bash: python3 station_means.py ...`	a file listing showing the new chart
9	**No tool call: the final answer.** A table of means and a note on the missing value.	

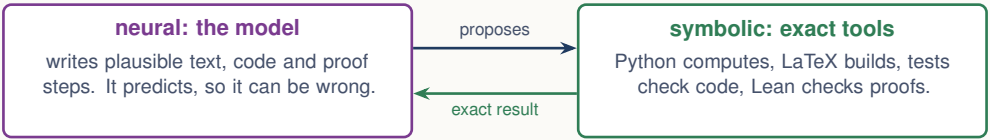
What to notice in that transcript

- **Every step was a tool call** until the final answer.
- **It looked before it acted** (step 2) and found the empty cell.
- **It checked its own output**: it read the chart, saw Amsterdam (13.63) and Delft (13.55) both labelled “13.6”, and fixed the labels.
- **The harness shaped the run**: a locally installed skill was loaded first.
- **Cost**: 10 turns, 26 s, about 192k input tokens (mostly cached re-reads), 1.9k output tokens. Reported cost $0.26.
- **Why know this?** Knowing how it works lets you give more precise instructions, so you get less output you don’t need.



The chart the agent produced

Neural + symbolic: the model proposes, tools check



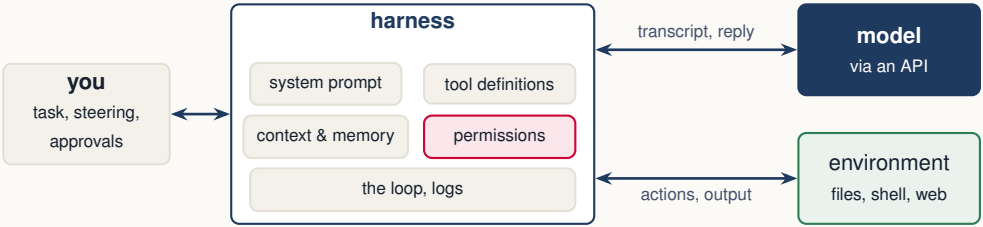
- In the transcript (steps 4 and 5), the agent computed the means by writing and running a Python script. That is the symbolic side.
- 2023: GPT-4 got 59% of 3-by-3-digit multiplications right without tools. Accuracy fell towards zero as numbers grew.
- Neuro-symbolic systems, such as AlphaGeometry, combine both. Checking happens in tests, scripts and Lean (“Autonomy, earned by verification”).

Dziri et al. (2023); Trinh et al. (2024); Lean: de Moura and Ullrich (2021).

0:48–0:55

The moving parts

The parts of an agent system



- The model usually runs on the provider's servers. Everything in the transcript is sent there.
- The harness and the environment usually run on your machine (or a cloud VM for cloud agents).
- An agent is a whole system: sandbox, harness, inference, training and monitoring surround the model.

System view after CMU 11-768 Lecture 1 (Fried and Neubig, 2026).

Same model, different harness, different behaviour

What the harness decides

- Which tools exist, and how they are described
- The system prompt and any instruction files it loads
- What it keeps, drops or summarises from the transcript
- What it remembers between sessions (memory files it loads)
- What runs without asking you
- When to stop

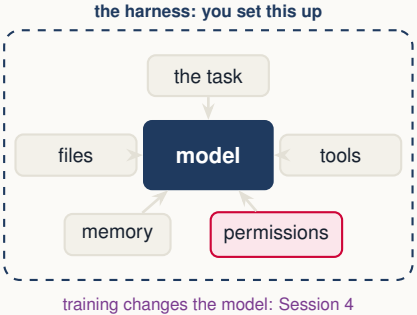
Evidence

- SWE-agent: the better equipped the harness (tools built for the agent, with clear feedback), the better the same model performed.

When an agent misbehaves, ask two questions: is this the *model*, or the *harness* (what it was given, allowed, told)?

Yang et al. (2024).

You are a harness engineer



- As users, we cannot retrain the model. We set up everything around it: the task, the files, the tools, the permissions, the memory.
- For this course, focus on what you can specify. The task, the files, the tools, the permissions, the memory.
- Point it to the source: it reads only the section it needs.
- How memory works: after the break.

Every task, file, tool, permission and memory file you set up is harness engineering.

"Two ways to build these capabilities" (LLM training, harness engineering), CMU 11-768 Lecture 1 (Fried and Neubig, 2026).

Break

0:55–1:05 back at 11:35

1:05–1:13

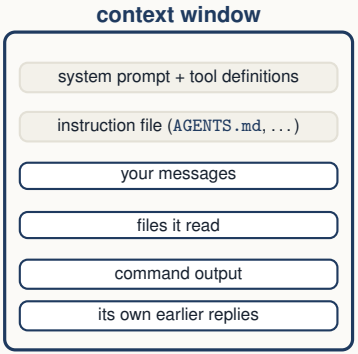
Mental models

Eight mental models

1 The context window is its entire world	**2** No memory between sessions unless written to files	**3** It sees only what a tool call brought in	**4** Long contexts degrade
5 Same input, different output	**6** Skills are uneven	**7** Confidently wrong, and agreeable	**8** Tokens are cost and time

Each one predicts behaviour you will see in your own work.

1–3: the context window is the agent's entire world



- Each call, the model sees exactly the transcript the harness sends. Nothing else exists for it.
- A new session starts empty, except what the harness loads from files.
- It sees only what a tool call brought in, such as the files it read. "Look at the plot" works only after the plot was read.
- Memory between sessions comes from files: next slide.

Instruction files: Anthropic (2026f); OpenAI (2026a); Google (2026a). Context engineering in depth: Session 2.

How agents remember: memory is files



Instruction files you write

`CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, in your project or home folder. Claude Code also loads them from folders above.

Memory files the tool writes

Claude Code's auto memory is on by default: an index loads every session. Review it with `/memory`. Codex memories are off by default.

Notes you ask it to keep

A `notes.md` or progress file in your project. The hands-on project has one.

The model remembers nothing. Files do. If it matters next time, write it to a file.

Anthropic (2026f); OpenAI (2026a); OpenAI (2026c); Google (2026b). Checked 3 Nov 2026.

4: long contexts degrade

Lost in the middle

- Models answered questions about documents in their context.
- Accuracy was often highest when the relevant information was at the start or end, and dropped when it was in the middle.

Context rot

- 18 models, simple tasks, only the input length varied.
- Performance became less reliable as the input grew, even though the task stayed the same.

A long agent session is a long context. Symptom: it fixates on an earlier idea, because it re-reads it every turn. Start a fresh session with a short summary (S3).

Liu et al. (2023); Hong et al. (2025).

5–7: nondeterministic, uneven, confident

Same input, different output

- Tokens are sampled, so reruns differ.
- Even at temperature 0, LLM APIs are not deterministic in practice: results depend on server batching.

Skills are uneven

- In a study of consultants using GPT-4, AI helped on some tasks and hurt on another.
- For agents: check each kind of task. One success does not carry over.

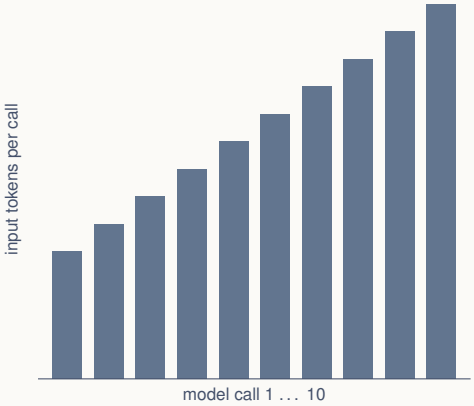
Confidently wrong, agreeable

- Training and evaluation reward guessing over saying “I don’t know”.
- Assistants trained on human feedback tend to agree with the user’s stated view (sycophancy).

Practical rule: ask for evidence you can check, such as a command output, a test or a page number.

He and Thinking Machines Lab (2025); Dell’Acqua et al. (2023); Kalai et al. (2025); Sharma et al. (2023). Causes in training: Session 4.

8: tokens are cost and time



Shape only: each call re-reads the growing transcript.

- The transcript example: about 192k input tokens for a 9-row CSV, because every call re-reads everything so far.
- Caching makes re-reads cheaper. They still cost tokens.
- Long sessions, big files and verbose command output all cost tokens, and time.
- Subscriptions hide this until you hit a usage limit. API use bills per token. (S2)

1:13–1:20

Before the hands-on: safe settings

Where agents run

Terminal (CLI)	Editor (IDE)	Cloud
runs on your machine, with shell access governed by permissions e.g. Claude Code, Codex CLI, Gemini CLI, Antigravity CLI (`agy`), mini-swe-agent	the same agent inside your editor, with inline diffs e.g. Claude Code for VS Code and JetBrains, Codex IDE extension	works on a clone of your repository in a remote VM, and you review the result e.g. Claude Code on the web, Codex Cloud, Jules

- Same loop everywhere. What differs: which environment it acts on, what it can see, who approves actions.
- **Gemini users:** since 18 June 2026 Gemini CLI no longer serves free, Google AI Pro or Ultra personal accounts. It still works with a paid Gemini API key or an organisation's Code Assist licence. Google's successor is Antigravity CLI.
- **Codex users:** the CLI needs ChatGPT Plus or higher, or an API key.

Anthropic (2026g); Anthropic (2026e); OpenAI (2026d); Lyalin and Mullen (2026); Google (2026d). Checked 26 Oct and 3 Nov 2026.

Permission modes: who approves an action?

`default` (Manual)



who approves: **you**

Reads run freely. It asks before most edits, commands and network access. We use it today.

`auto`



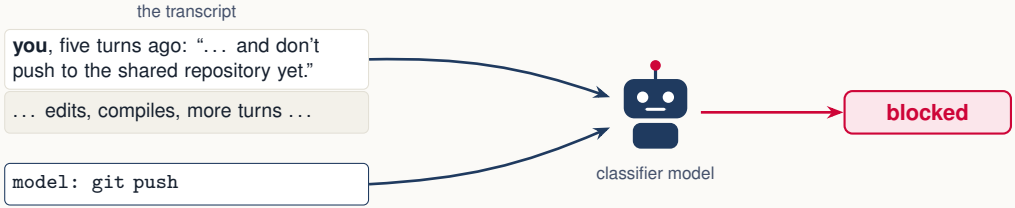
who approves: **a second model**

Rules and safe actions first. A classifier model reviews the riskier rest. The default in terminal sessions since v2.1.283, when available. Next slide.

Today: `claude --permission-mode default`. Other modes and other tools: Session 2.

Anthropic (2026h); Anthropic (2026c). Checked 3 Nov 2026.

Auto mode: a second model reviews risky actions



- Rules and safe actions first. Riskier actions go to a second model.
- It reads the conversation: if you said “don’t push”, it blocks the push.
- It can forget after a long session. For a hard limit, use a rule or a sandbox (Session 2).

Anthropic (2026c). Checked 27 Oct 2026.

Before you start: your data and training



Anthropic (Claude Code)

Free, Pro, Max: turn off model improvement in the privacy settings. Team, Enterprise and API: not used for training by default. Feedback you share has separate terms.



OpenAI (ChatGPT, Codex)

Free, Plus, Pro: *on* by default. Turn off “Improve the model for everyone” (Data controls). It covers Codex tasks. Separately, turn off Codex’s “Include environments” in Codex settings. The ChatGPT control does not change it. Business, Enterprise, Edu, API: off by default.



Google (Antigravity, personal account)

Your interactions are used to improve Google products and may be read by Google reviewers. Opt out of this use in Antigravity settings.

Anthropic (2026b); OpenAI (2026e); OpenAI (2026f); Google (2026c). Checked 27 Oct and 3 Nov 2026. Terms change.

1:20–1:50

Hands-on

Hands-on: from install to a first result



Install and log in

8 min

Claude Code, in a cautious mode



Your first project

10 min

a local LaTeX folder and a PDF



Your first prompt

8 min

live demo: a new idea, written up



Take-home task

4 min

choose one. We start Session 2 with it.

- We do each step live on screen. Follow along on your laptop, or just watch.
- Already set up? Help a neighbour. Self-study exercises are in the appendix.

Install and log in: Claude Code

Claude Code (Anthropic: Pro plan or higher)

macOS, Linux, WSL	`curl -fsSL https://claude.ai/install.sh | bash`
Windows PowerShell	`irm https://claude.ai/install.ps1 | iex`
Log in	new terminal: run `claude`, follow the browser prompt, then `/exit`
Test	`claude --version` and `claude auth status --text`
Start safely	`claude --permission-mode default` (Manual), in every new session
Training off	once, in your Claude privacy settings (“Before you start”)

Codex CLI (OpenAI, 2026b) also works for this course. Its install guide was linked in the Session 1 email.

One agent is enough. Already set up? Help a neighbour.

Anthropic (2026i); Anthropic (2026c). Checked 27 Oct 2026.

Your first project

In a terminal, go to the folder where you keep your work, start Claude Code in Manual mode, and type:

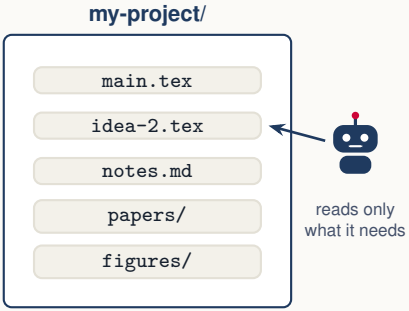
PROMPT

```
Download https://bilgehan.io/latex.md read the whole file, and follow it to set up my first LaTeX project.
```

1. checks if LaTeX is installed
2. asks before installing anything
3. creates main.tex, notes.md
4. compiles and opens main.pdf
5. saves a `/new-project` skill

Checkpoint. A PDF with a small figure opens on your screen. Approve only the actions you understand. If you are unsure, ask us.

Why a local LaTeX or Markdown folder?



- **A clean context.** The agent reads only the files it needs (mental models 1 and 4).
- **It can search.** It finds things in your files itself, for example with `grep`.
- **Notes live in files.** `notes.md` survives a new session.
- **Markdown, LaTeX or both.** We usually work in both.
- **A new .tex file for each new idea**, so work does not get mixed up.

Your first prompt: a live demo

A first real task in your new project, as a pattern:

PROMPT

```
Create a new file idea.tex. Study the paper in papers/. Propose something new: an
improvement, a generalisation or a counterexample. Write it up with the statement, a
proof or argument, and a figure in TikZ. Compile it and fix any errors.
```

Then iterate, for example:

PROMPT

```
The figure does not show the key idea. Make a clearer figure, and explain in two
sentences what it shows.
```

It is never one-shot: check the result, then ask again. A convincing write-up is not a proof you have checked.

Your take-home task: choose one

Senior researchers
Ask the question you have always wanted answered.

Students and postdocs
Take a paper you did not understand, and work through it with the agent.

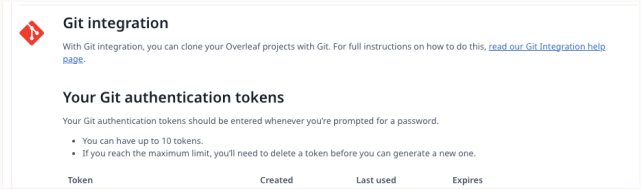
Or, for example

- a web page for your project
- a mathematical analysis, with a proof
- a related-work document
- an explanation of a paper
- a data analysis

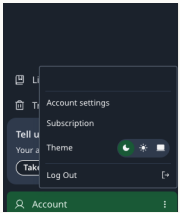
Write down what you asked and what you checked.

Checkpoint. We start Session 2 with a few of your tasks: what you asked, what happened, and what you checked.

Optional: link to Overleaf with git



Overleaf: Account settings, Git integration



the account menu

- If you want, you can sync your project with Overleaf through git. Your agent can help you set it up: just ask it.
- Git integration is a premium Overleaf feature.
- A token works like a password for all your projects.

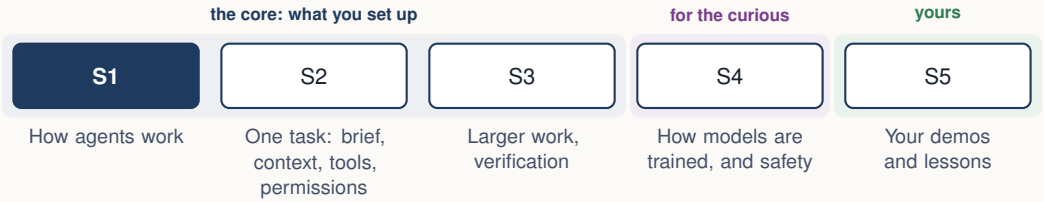
1:50–2:00

Wrap-up

Today in five sentences

1. A chat is a stateless model applied to a transcript. An **agent** is that model in a loop with tools.
2. The loop: **model** writes a tool call $\rightarrow$ **harness** checks and executes it $\rightarrow$ the **observation** goes back into the transcript $\rightarrow$ model.
3. An agent system has five parts: **model, harness, tools, environment, you**. As users, we work on the harness: task, files, tools, permissions, memory.
4. The context window is its whole world: it forgets between sessions, degrades when long, varies between runs, and can be confidently wrong.
5. There is no oracle for autonomy: with a robust way to check, you can let it run.

The course from here



Verification and responsibility run through all five sessions.

Next time: working effectively

Session 2 · Tuesday 10 November 2026 · 10:30–12:30 · Euler room, CWI

- Writing a task: context, constraints, a definition of “done”
- Instruction files: `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`
- Tools and extensions: MCP, skills
- Steering, interrupting, recovering
- Permissions, sandboxing, prompt injection, what leaves your machine
- We revisit today’s sorting exercise

Before S2: do your take-home task, and note what you checked. We start Session 2 with it.

Readings

Core

- Anthropic, *Building Effective AI Agents* (Schluntz and Zhang, 2024)
- Yao et al., *ReAct* (Yao et al., 2022)

Optional

- Schick et al., *Toolformer* (Schick et al., 2023)
- mini-swe-agent, the code (SWE-agent team, 2026)
- Kwa et al. (METR), *Measuring AI Ability to Complete Long Software Tasks* (Kwa et al., 2025)
- Russell & Norvig, *AIMA*, ch. 2 (Russell and Norvig, 2021)

Full references follow. All links are in the PDF.

References I

Anthropic. Prompt caching. Claude API documentation, accessed 27 Oct 2026, 2026a. URL https://platform.claude.com/docs/en/build-with-claude/prompt-caching.

Anthropic. Data usage. Claude Code documentation, accessed 26 Oct 2026, 2026b. URL https://code.claude.com/docs/en/data-usage.

Anthropic. Choose a permission mode. Claude Code documentation, accessed 27 Oct 2026, 2026c. URL https://code.claude.com/docs/en/permission-modes.

Anthropic. Best practices for Claude Code. Claude Code documentation, 2026d. URL https://code.claude.com/docs/en/best-practices.

Anthropic. Claude Code documentation. Pages on memory, sessions, checkpointing, MCP, skills, sandboxing, costs; accessed 26 Oct 2026, 2026e. URL https://code.claude.com/docs.

Anthropic. How Claude remembers your project. Claude Code documentation, accessed 26 Oct 2026, 2026f. URL https://code.claude.com/docs/en/memory.

Anthropic. Claude Code overview. Claude Code documentation, accessed 26 Oct 2026, 2026g. URL https://code.claude.com/docs/en/overview.

Anthropic. Configure permissions. Claude Code documentation, accessed 26 Oct 2026, 2026h. URL https://code.claude.com/docs/en/permissions.

References II

Anthropic. Advanced setup. Claude Code documentation, accessed 27 Oct 2026, 2026i. URL https://code.claude.com/docs/en/setup.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021. URL https://doi.org/10.1007/978-3-030-79876-5_37.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

Fabrizio Dell'Acqua, Edward McFowland III, Ethan R. Mollick, Hila Lifshitz-Assaf, Katherine Kellogg, Saran Rajendran, Lisa Krayer, François Candelon, and Karim R. Lakhani. Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. Working Paper 24-013, Harvard Business School, 2023. URL https://doi.org/10.2139/ssrn.4573321.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, et al. Faith and fate: Limits of transformers on compositionality. *arXiv preprint arXiv:2305.18654*, 2023. URL https://arxiv.org/abs/2305.18654.

Daniel Fried and Graham Neubig. 11-768 AI agents. Carnegie Mellon University course, Fall 2026, 2026. URL https://www.cmu-agents.com/.

References III

GitHub. Setting repository visibility. GitHub Docs, accessed 3 Nov 2026, 2026. URL https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility.

Google. Gemini CLI configuration. Gemini CLI documentation, accessed 26 Oct 2026, 2026a. URL https://geminicli.com/docs/reference/configuration.

Google. Provide context with GEMINI.md files. Gemini CLI documentation, accessed 3 Nov 2026, 2026b. URL https://geminicli.com/docs/cli/gemini-md/.

Google. Google antigravity additional terms of service. Accessed 26 Oct 2026, 2026c. URL https://antigravity.google/terms.

Google. Jules: Getting started. Jules documentation, accessed 26 Oct 2026, 2026d. URL https://jules.google/docs/.

Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. How we built our multi-agent research system. Anthropic, June 2025. URL https://www.anthropic.com/research/multi-agent-research-system.

Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma Technical Report, July 2025. URL https://www.trychroma.com/research/context-rot.

References IV

Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*, 2025. URL https://arxiv.org/abs/2509.04664.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*, 2022. URL https://arxiv.org/abs/2205.11916.

Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, et al. Measuring AI ability to complete long software tasks. *arXiv preprint arXiv:2503.14499*, 2025. URL https://arxiv.org/abs/2503.14499.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023. URL https://arxiv.org/abs/2307.03172.

Dmitry Lyalin and Taylor Mullen. An important update: Transitioning Gemini CLI to Antigravity CLI. Google Developers Blog, May 2026. URL https://developers.googleblog.com/an-important-update-transitioning-gemini-cli-to-antigravity-cli/.

OpenAI. OpenAI o1 system card. *arXiv preprint arXiv:2412.16720*, 2024. URL https://arxiv.org/abs/2412.16720.

OpenAI. Custom instructions with AGENTS.md. Codex documentation, accessed 26 Oct 2026, 2026a. URL https://learn.chatgpt.com/docs/agent-configuration/agents-md.

OpenAI. Codex CLI. Codex documentation, accessed 27 Oct 2026, 2026b. URL https://learn.chatgpt.com/docs/codex/cli.

OpenAI. Memories. Codex documentation, accessed 3 Nov 2026, 2026c. URL https://learn.chatgpt.com/docs/customization/memories.

References V

OpenAI. Codex pricing. Codex documentation, accessed 26 Oct 2026, 2026d. URL https://learn.chatgpt.com/docs/pricing.

OpenAI. Data controls in ChatGPT. OpenAI Help Center, accessed 26 Oct 2026, 2026e. URL https://help.openai.com/en/articles/7730893-data-controls-in-chatgpt.

OpenAI. How your data is used to improve model performance. OpenAI Help Center, accessed 26 Oct 2026, 2026f. URL https://help.openai.com/en/articles/5722486-how-your-data-is-used-to-improve-model-performance.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2021. URL https://aima.cs.berkeley.edu/. Chapter 2, Intelligent Agents.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. URL https://arxiv.org/abs/2302.04761.

Erik Schluntz and Barry Zhang. Building effective AI agents. Anthropic, December 2024. URL https://www.anthropic.com/research/building-effective-agents.

Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, et al. Towards understanding sycophancy in language models. *arXiv preprint arXiv:2310.13548*, 2023. URL https://arxiv.org/abs/2310.13548.

SWE-agent team. mini-swe-agent: The minimal AI software engineering agent. GitHub repository, version 2, commit 04d809c (3 Sep 2026), 2026. URL https://github.com/SWE-agent/mini-swe-agent.

References VI

Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625: 476–482, 2024. URL https://www.nature.com/articles/s41586-023-06747-5.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *arXiv preprint arXiv:2305.04388*, 2023. URL https://arxiv.org/abs/2305.04388.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, et al. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022. URL https://arxiv.org/abs/2201.11903.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL https://arxiv.org/abs/2210.03629.

Appendix

Self-study exercises · Background · mini-swe-agent, a minimal agent

Self-study starter exercises

1 Orientation	2 See the loop	3 Statelessness	4 Catch the error
5 min	8 min	5 min	8 min
Which tools does it call?	Model step or tool step?	What survives a new session?	What did *you* check?

- Four short exercises to do on your own, in this order. Each ends with a **checkpoint**.
- Materials: folder `s1/` of the course exercise repository, or files of your own (each exercise says which).
- Work in a new folder with copies, in Manual mode (Claude Code: `claude --permission-mode default`), and read every approval prompt.

Exercise 1: orientation (5 min)

Setting: a small project you have never seen, with code, a data file, and a paper PDF: folder `s1/` of the course exercise repository, or any small unfamiliar repository.

PROMPT

```
Explain what this project does, how the code, the data and the paper relate to each other, and how I would run it. Do not change any files.
```

Observe

- Which tools did it call, in which order? Did it open the PDF, or guess from file names?
- Did it ask permission for anything? For what?
- Is there a claim in its explanation you could check in 30 seconds? Check one.

Checkpoint. You can name three tool calls it made, and one claim you verified or refuted.

Exercise 2: see the loop (8 min)

Setting: a CSV file with numeric columns, from `s1/` of the course exercise repository, or your own.

PROMPT

```
From the CSV file in this folder, compute the mean and standard deviation of each
numeric column. Then plot them as a bar chart with error bars and save the figure.
Save the analysis code as a script file and run it.
```

Annotate the transcript, like the example we read:

- For each step: *model wrote* or *harness ran*?
- What did each observation tell the model?
- Did it look at its own figure?

#	model or tool?	what it saw
1	e.g. tool	first lines of the CSV
2		

Checkpoint. The saved script and the figure exist, and you have checked a number in the figure yourself, by hand or with your own one-line command.

Exercise 3: statelessness (5 min)

1. Quit the agent. Start a **fresh** session in the same folder (do not use a resume or continue option).
2. Ask:

   PROMPT

   ```
   What did we do in the previous session? What value did you compute?
   ```

3. Tell this fresh session what you did and the result you checked, then ask it to write those facts to `NOTES.md`. Quit and start another fresh session. Ask:

   PROMPT

   ```
   Read NOTES.md. What did we do, and what result did we obtain?
   ```

Observe: what changed between the two fresh sessions? If it “remembered” in step 2, find out which file it read: some tools write their own memory files.

Checkpoint. You can point to the file that carried information from one session to the next.

Exercise 4: catch the error (8 min)

Setting: a copy of your script from Exercise 2. Plant one small error in it yourself, for example drop one row before computing the means.

1. **Unprompted** (did it notice anything?):

   PROMPT

   ```
   Run this analysis and summarise the result.
   ```

2. **Prompted:**

   PROMPT

   ```
   Review this analysis for errors. Show me the evidence for each issue.
   ```

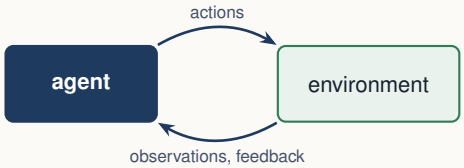
3. **You:** what did you have to check yourself before you would trust its answer?

Checkpoint. The agent found your error unprompted, when asked, or not at all, and you know what evidence you checked.

What is an agent? The precise version

> *"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."*

Russell and Norvig (2021), ch. 2



Environment	a repository, a folder of papers, a spreadsheet or data folder
Observations	file contents, command output, web pages, error messages
Actions	shell commands, file edits, web requests, messages to you
Feedback	tests pass, the script runs, you say "no, not that"

Framing after Russell and Norvig (2021) and CMU 11-768 Lecture 1 (Fried and Neubig, 2026).

The same frame, four research settings

	Code repository	Paper folder	Data folder	Proof
Observes	source files, test output, git diff	PDFs, notes, `.bib` files	CSV files, script output, figures	the statement, definitions, draft proof
Acts	edits files, runs tests, commits	extracts, compares, drafts text	writes and runs analysis code, makes plots	writes proof steps or Lean code
Feedback	tests pass or fail	do the citations exist and say that?	the numbers check out, the figure looks right	Lean checks it. Otherwise you check each step
Watch out	edits tests to pass	invented references	silently drops or changes rows	proves a different statement than you meant

The *Feedback* row is the verifier from "Autonomy, earned by verification": where it is automatic, the loop can run further, and where it is you, it should ask first.

Lean: de Moura and Ullrich (2021).

A map of agent capabilities

Accurate tool calling — S1
today, and how it is trained: S4

Coherence over long context — S3
long sessions, compaction

Customisability — S2
instruction files, MCP, skills

Complex task management — S3
decomposition, subagents

Environment understanding — S2
giving it the right context

Safety — S2, S4
permissions, injection (S2), research view (S4)

Each one improves through training the model (Session 4), or through the harness around it. The harness is your part.

Capability taxonomy from CMU 11-768 Lecture 1 (Fried and Neubig, 2026). Mapping to our sessions is ours.

mini-swe-agent: a minimal agent

- From the Princeton and Stanford team behind SWE-bench and SWE-agent.
- **One tool:** bash. No file editor, no search tool. The model uses the shell.
- **Linear history:** every step appends to the message list. The trajectory *is* the prompt.
- **Independent actions:** each command runs in a fresh `subprocess`, so swapping in a sandbox (Docker, Singularity) is one line.

Where the parts live (`src/minisweagent/`)

`agents/default.py`	the loop
`models/litellm_model.py`	calls the LLM
`environments/local.py`	runs bash
`config/mini.yaml`	the prompts

Claude Code and Codex add many tools, permissions and context management. The core is this loop.

SWE-agent team (2026), `src/minisweagent/`. Repository read on 26 Oct 2026 (v2, commit 04d809c).

The loop in code

```python
class DefaultAgent:
    def run(self, task):
        self.messages = [system_msg, task_msg(task)]
        while self.messages[-1]["role"] != "exit":
            try:
                self.step()
            except InterruptAgentFlow as e:  # e.g. Submitted
                self.add_messages(*e.messages)
    def step(self):
        return self.execute_actions(self.query())
    def query(self):  # also stops on step, cost, time limits
        message = self.model.query(self.messages)
        self.add_messages(message)
        return message
    def execute_actions(self, message):
        actions = message["extra"]["actions"]
        outputs = [self.env.execute(a) for a in actions]
        return self.add_messages(*self.model
            .format_observation_messages(message, outputs))
```

- `model.query` gets the *whole* message list every time: statelessness, in code.
- The model's reply is parsed into `actions` (bash commands).
- `env.execute` runs them, and outputs come back as new messages.
- Finishing raises `Submitted`, an `InterruptAgentFlow`. `run` appends its `exit` message, which ends the loop.

Simplified from `agents/default.py` (SWE-agent team, 2026). Other error handling, templates, cost tracking, saving removed.

Running the command, and stopping

```python
class LocalEnvironment:
    def execute(self, action):
        command = action.get("command", "")
        result = _run(command, cwd, env, timeout)
        #        subprocess.Popen(command, shell=True, ...)
        output = {"output": result.stdout,
                  "returncode": result.returncode}
        self._check_finished(output)
        return output

# next observation (config/mini.yaml):
# {"returncode": 0, "output": "..."}
```

- The model's text runs as a shell command. No permission check here.
- The task ends when the model runs `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`.
- Swap `LocalEnvironment` for a Docker one and the same loop runs in a container.

Running an agent means letting model-written text become commands. Where that happens is your choice.

Simplified from `environments/local.py` and `models/litellm_model.py` in SWE-agent team (2026) (MIT licence).

Code attribution

The mini-swe-agent excerpts on the “Step 4”, “The loop in code” and “Running the command” slides are adapted from `https://github.com/SWE-agent/mini-swe-agent` at commit `04d809c`, under the following licence.

MIT License

Copyright (c) 2025 Kilian A. Lieret and Carlos E. Jimenez

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.